🚀 AWS CI/CD Pipeline with Automated Deployment

> End-to-end CI/CD pipeline integrating GitHub with AWS CodePipeline, CodeBuild, and Elastic Beanstalk — featuring automated build validation, auto-provisioning, and real-time failure alerting via CloudWatch & SNS.

📸 Screenshots

1. CodePipeline — Successful Deployment

All stages green: Source → Build → Deploy

2. Elastic Beanstalk — Live Application

Flask app running on Elastic Beanstalk with auto-provisioned EC2

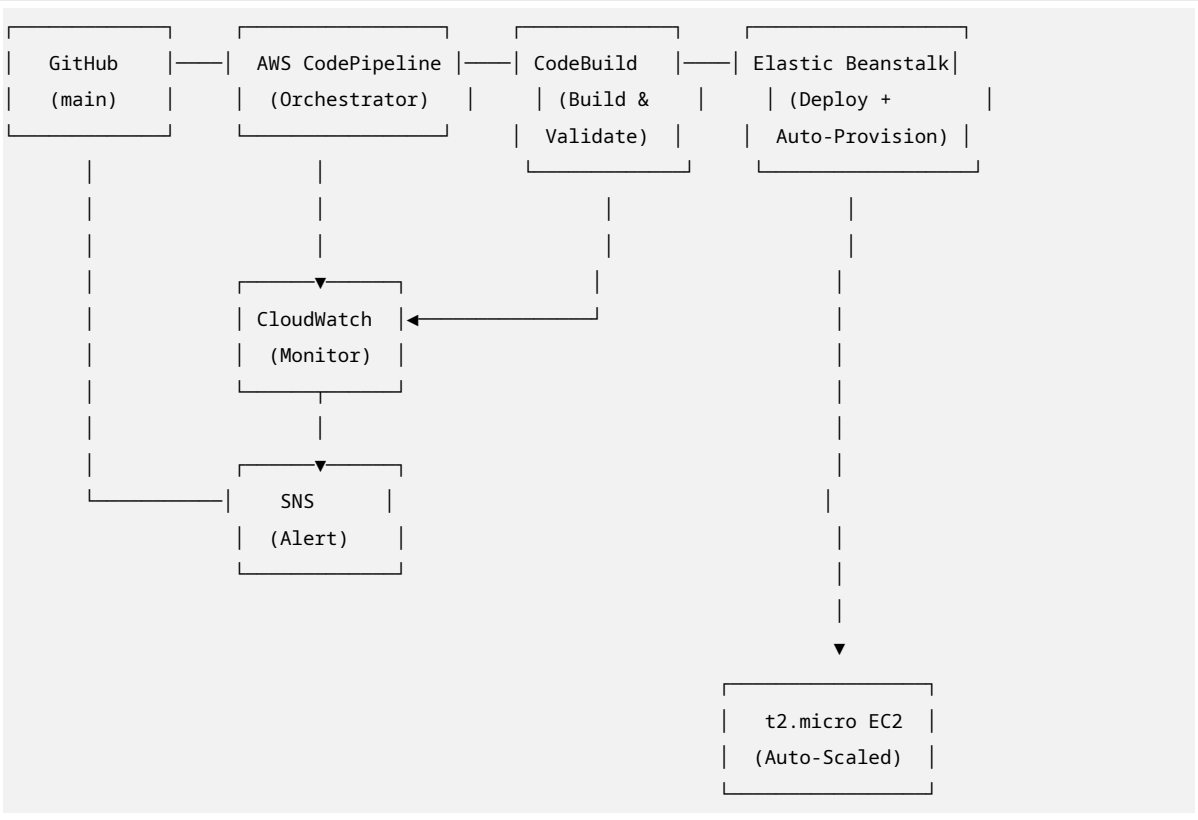
3. CloudWatch Alarm — Failure Detection

Alarm monitoring pipeline ExecutionFailed metric

4. SNS Email Alert — Real-time Notification

Email alert received within 2 minutes of pipeline failure

🏗️ Architecture



Pipeline Flow

Stage	Service	What Happens
1. Source	GitHub (via CodeConnections)	Detects push to `main` branch, triggers pipeline automatically
2. Build	AWS CodeBuild	Installs dependencies, validates Python syntax, imports application, packages artifacts
3. Deploy	Elastic Beanstalk	Deploys Flask app, auto-provisions EC2 instance, starts Gunicorn + Nginx
4. Monitor	CloudWatch + SNS	Tracks `ExecutionFailed` metric, sends email alert on failure

---

🛠️ Tech Stack

Category	Technology
Source Control	GitHub
CI/CD Orchestration	AWS CodePipeline
Build Service	AWS CodeBuild
Compute Platform	AWS Elastic Beanstalk (Python 3.12)
Web Framework	Flask
WSGI Server	Gunicorn
Reverse Proxy	Nginx (managed by EB)
Monitoring	Amazon CloudWatch
Alerting	Amazon SNS (Email)
IAM	AWS IAM Roles & Policies

📁 Project Structure

```
my-cicd-demo/
├── application.py          # Flask application
├── requirements.txt        # Python dependencies (Flask, Gunicorn)
├── Procfile                # EB process definition (web: gunicorn...)
├── buildspec.yml           # CodeBuild instructions
└── README.md               # This file
```

⚡ Key Features

1. Automated Trigger

- Push code to GitHub `main` branch → Pipeline starts automatically within 10-20 seconds
- No manual intervention required

2. Build Validation

- `buildspec.yml` validates code before deployment:
  - `python -m py_compile application.py` — syntax check
  - `python -c "from application import application"` — import check
- **Broken code never reaches production**

3. Auto-Provisioning

- Elastic Beanstalk automatically provisions EC2 instances
- No manual server setup or configuration

4. Failure Detection & Alerting

- CloudWatch alarm monitors `ExecutionFailed` metric
- SNS sends email alert within 2 minutes of failure
- Improves Mean Time To Detect (MTTD)

🚀 How to Deploy

Prerequisites

- AWS Account (Free Tier eligible)
- GitHub Account

Step 1: Create Elastic Beanstalk Environment

1. AWS Console → Elastic Beanstalk → Create Application
2. Platform: Python 3.12
3. Environment type: Single instance
4. Instance type: t2.micro (Free Tier)

Step 2: Create CodeBuild Project

1. AWS Console → CodeBuild → Create project
2. Source: GitHub (connect via CodeConnections)
3. Buildspec: Use `buildspec.yml` from repo

Step 3: Create CodePipeline

1. AWS Console → CodePipeline → Create pipeline
2. Source: GitHub (Version 2)
3. Build: CodeBuild project
4. Deploy: Elastic Beanstalk environment

Step 4: Configure CloudWatch + SNS

1. CloudWatch → Alarms → Create alarm
2. Metric: CodePipeline `ExecutionFailed`
3. Action: Send notification to SNS topic
4. SNS → Create email subscription

🧪 Testing the Pipeline

Test 1: Successful Deployment

```
# Edit application.py and push
git add .
git commit -m "Update homepage message"
git push origin main

# Pipeline auto-triggers and deploys
```

Test 2: Failure Detection

```
# Intentionally break the code (e.g., delete Flask import)
git add .
git commit -m "Test failure scenario"
git push origin main

# Build stage fails → CloudWatch alarm fires → Email alert sent
```

📊 Results

Metric	Before	After
Deployment Time	Manual (~30 min)	Automated (~5 min)
Failure Detection	Manual checking	Automatic (< 2 min)
Server Management	Manual provisioning	Auto-provisioned by EB

🎓 What I Learned

- **AWS CodePipeline V2** with GitHub App integrations and webhook triggers
- **IAM Policy Management** — attaching least-privilege policies to service roles
- **Elastic Beanstalk internals** — Procfile, platform hooks, nginx proxy configuration
- **CloudWatch Metrics** — monitoring custom pipeline metrics and setting thresholds
- **Troubleshooting** — debugging 502 errors, port mismatches, and IAM permission issues

---

🧹 Cleanup (Important!)

To avoid AWS charges, terminate resources after demo:

```
# Elastic Beanstalk
AWS Console → Elastic Beanstalk → Your Environment → Actions → Terminate

# Optional: Delete other resources
```

```
AWS Console → CodePipeline → Delete pipeline
AWS Console → CodeBuild → Delete project
AWS Console → CloudWatch → Delete alarm
AWS Console → SNS → Delete topic
```

> **Note:** Only the Elastic Beanstalk environment (EC2 instance) incurs charges. All other services used are within AWS Free Tier limits.

📫 Contact

- **GitHub:** @sneha7665
- **Project Repo:** sneha7665/my-cide-demo

Built with ☕ and a lot of CloudWatch logs.